

What is a Prototype?

A prototype is an object from which another object can inherit properties and methods.

OR

Every JavaScript object has an internal hidden link ([[Prototype]]) that points to another object called its prototype.

When a property or method is not found on the object itself, JavaScript automatically looks for it in the prototype.

Why it Exists : To enable Inheritance

1: Every object in JavaScript is just an object

Let's create a simple object.

```
const person = {  
  name: "Krunal",  
  age: 20  
};
```

You might think this object only contains two properties.

```
person{  
  name : "Krunal"  
  age  : 20  
}
```

But internally JavaScript creates something like this:

```
person{  
  name : "Krunal"  
  age  : 20  
  
  [[Prototype]]  
}
```

That hidden `[[Prototype]]` is what developers commonly inspect as `__proto__` (though `__proto__` itself is a legacy accessor).

So every object secretly contains a link to another object.

2: What is `__proto__` ?

Think of `__proto__` as a **pointer**.

- It doesn't store methods.
- It doesn't store data.

It only tells JavaScript:

"If you can't find something inside me, go look over there."

For example,

```
const person = {  
  name: "Krunal"  
};  
  
console.log(person.__proto__);
```

Output:

```
Object.prototype
```

Meaning

```
person  
  |  
  | __proto__  
  ▼  
Object.prototype
```

Notice something:

`person` itself doesn't have methods like

- `toString()`
- `hasOwnProperty()`
- `valueOf()`

Yet this works.

```
person.toString();
```

Why?

Because JavaScript couldn't find `toString()` inside `person`.

So it followed the `__proto__` link.

3: How JavaScript searches

Suppose

```
const student = {  
  name: "Krunal"  
};  
console.log(student.name);
```

Now

```
student.name
```

JavaScript checks

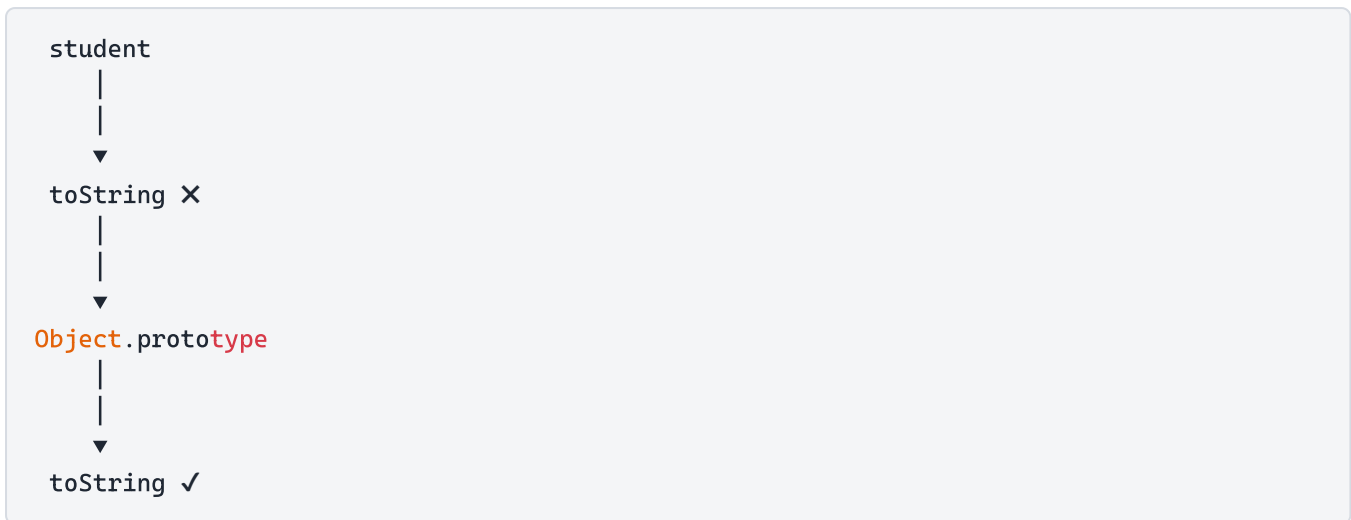
```
student  
|  
|  
▼  
name ✓
```

Done.

Now

```
student.toString()
```

JavaScript checks

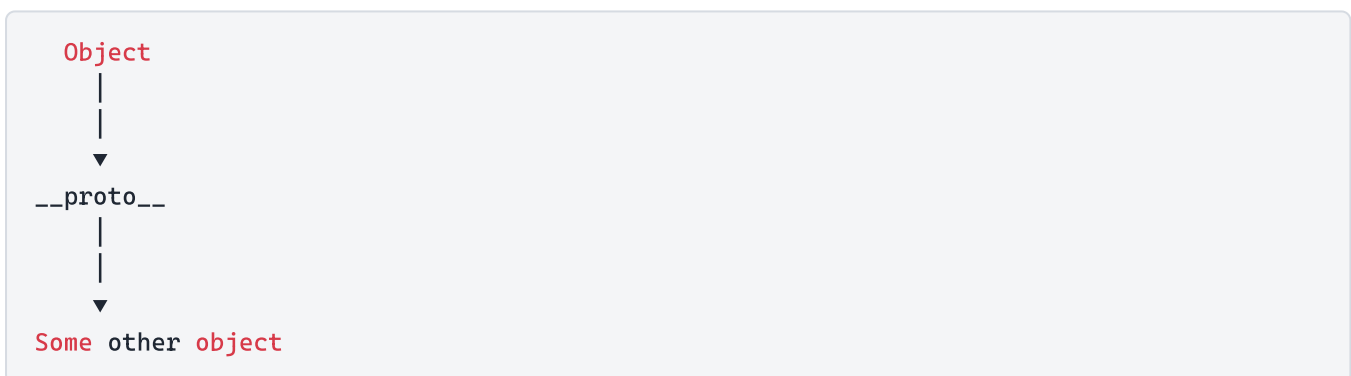


Found.

This searching process is called **Prototype Chain**

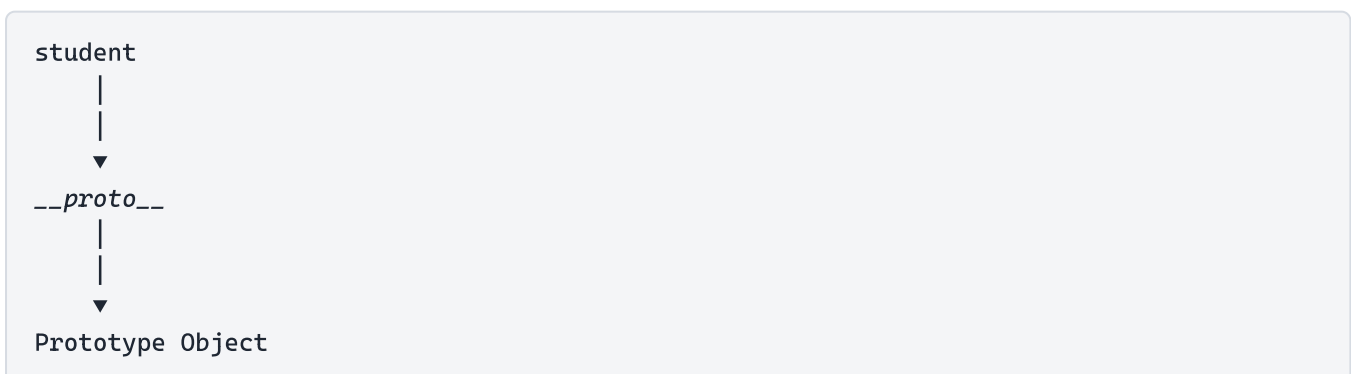
4: What exactly is Prototype?

Now we know



That "some other object" is called **Prototype**

So



In simple words

Prototype is the object that another object inherits from.

Notice the difference.

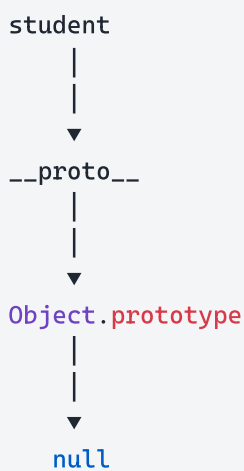
`__proto__`

- is the link

Prototype

- is the destination

Example



5: Where does this Prototype come from?

Now comes the interesting part.

Consider

```
function Student(name) {  
  this.name = name;  
}
```

Since functions are also objects, they have a special property called

`Student.prototype`

Let's print it.

```
console.log(Student.prototype);
```

Output

```
{}
```

JavaScript automatically creates it.

6: Why does every function have a prototype?

Imagine you're creating 500 students.

```
let s1 = new Student("A");  
let s2 = new Student("B");  
let s3 = new Student("C");
```

If every object stores

```
study()  
walk()  
talk()
```

inside itself

```
s1  
study()  
walk()  
talk()  
-----  
s2  
study()  
walk()  
talk()
```

Now imagine

10000 students.

That's 10000 copies.

Huge waste.

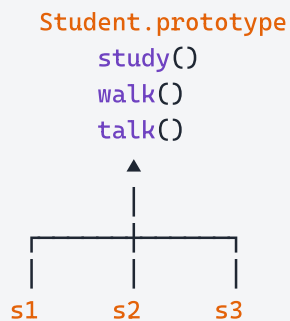
JavaScript says

"Why not keep one copy?"

So it creates

```
Student.prototype  
study()  
walk()  
talk()
```

Every student simply points to it.



Memory saved.

7: What does `new` actually do?

This is where everything connects.

When you write

```
let s1 = new Student("Krunal");
```

JavaScript internally does approximately this:

> Creates a new empty object

```
{}
```

> Sets its `__proto__`

```
newObject.__proto__  
  |  
  ▼  
Student.prototype
```

> Calls the constructor

```
Student.call(newObject, "Krunal");
```

> Returns the object

Final structure

```
s1  
name : Krunal  
  |  
  ▼  
__proto__  
  |  
  ▼  
Student.prototype  
  |  
  ▼  
Object.prototype  
  |  
  ▼  
null
```

This is why

```
s1 instanceof Student
```

returns

```
true
```

8: How do we use prototype?

Instead of writing

```
function Student(name) {  
  this.name = name;  
  
  this.study = function () {  
    console.log("Studying");  
  };  
}
```

write

```
function Student(name) {  
  this.name = name;  
}  
  
Student.prototype.study = function () {  
  console.log("Studying");  
};
```

Now

```
const s1 = new Student("A");  
const s2 = new Student("B");  
const s3 = new Student("C");
```

All of them share

```
study()
```

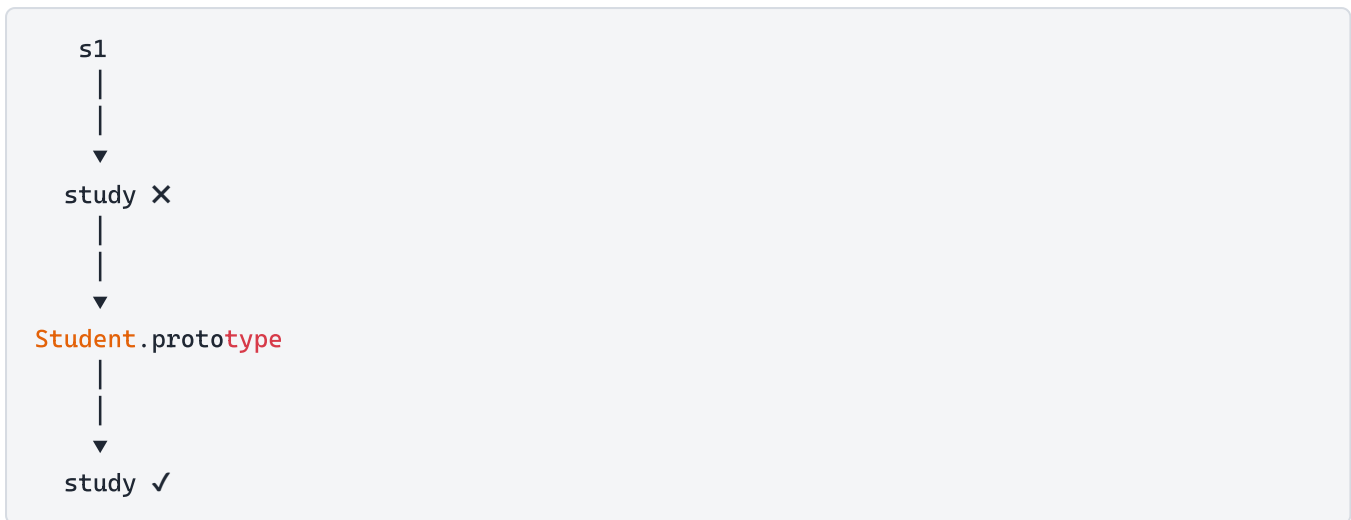
instead of making copies.

9: How does JavaScript find `study()` ?

Suppose

```
s1.study();
```

Search

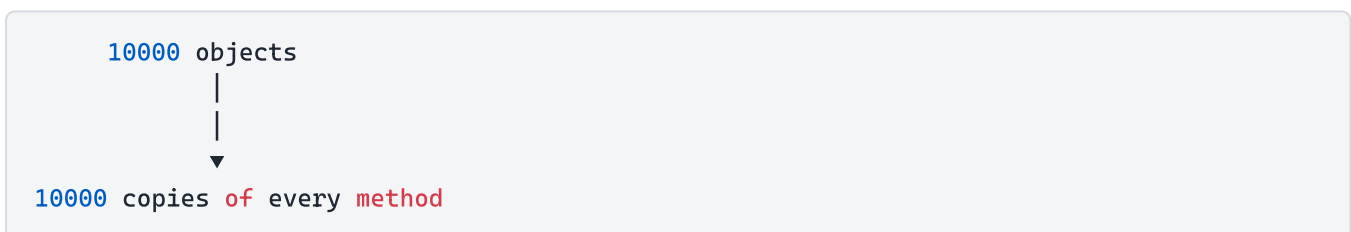


Found.

Executed.

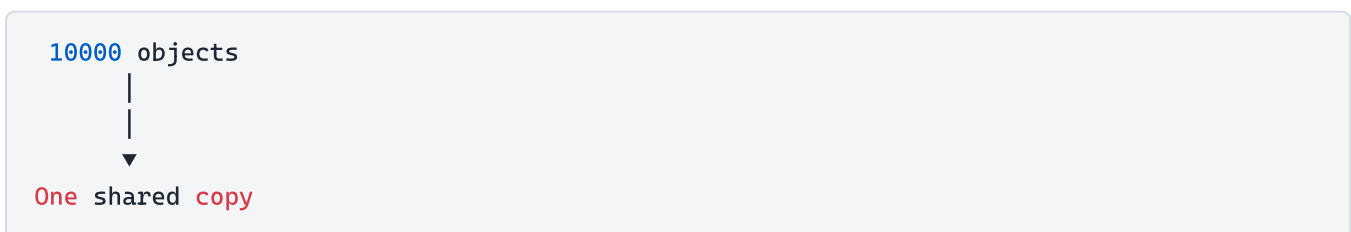
10: Why do we use prototypes?

Without prototype



Memory usage : 📈 **Very high**

With prototype



Memory usage : 📉 **Very low**

Benefits

- ✓ Saves memory by sharing methods.
 - ✓ Faster object creation because only data is stored on each instance.
 - ✓ Makes code easier to maintain—change one prototype method and every instance uses the updated version.
 - ✓ Enables inheritance through the prototype chain.
 - ✓ This is the foundation behind JavaScript classes.
-

11: How do ES6 Classes fit into this?

When you write

```
class Student {  
  constructor(name) {  
    this.name = name;  
  }  
  
  study() {  
    console.log("Studying");  
  }  
}
```

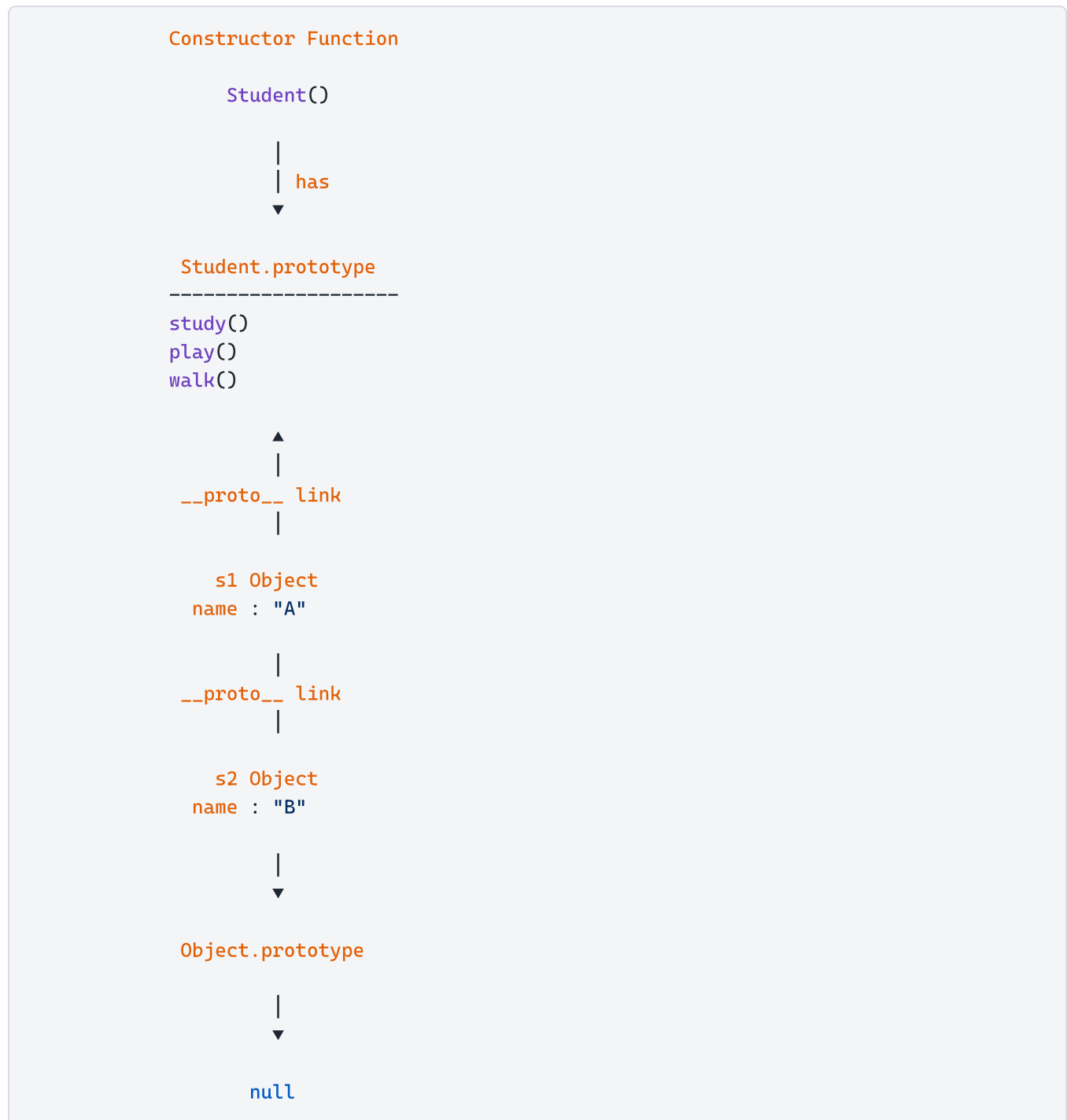
It **looks** different, but JavaScript still uses prototypes behind the scenes.

It is roughly equivalent to:

```
function Student(name) {  
  this.name = name;  
}  
  
Student.prototype.study = function () {  
  console.log("Studying");  
};
```

So **classes are syntactic sugar** over the prototype system—they provide a cleaner syntax but rely on the same underlying mechanism.

The Complete Picture



One sentence to remember

- **__proto__** → the hidden link from an object to another object.
- **prototype** → the object that constructor functions provide for their instances to inherit from.
- **Prototype chain** → JavaScript's lookup path when a property or method isn't found on the object itself.

- **Why it exists** → to enable inheritance and let many objects share one set of methods instead of duplicating them.